

École nationale supérieure des Mines de Saint-Etienne

LaTeXProject
Rapport

TD 3- Architecture des Processeurs

Élèves :
Samy TALEB
Ylies VEYRAT

Encadrants :
PONTIE Simon
GOUSSELOT Théophile

18 décembre 2024

Table des matières

1 Introduction et mise en contexte **3**

2 Modification du chemin de contrôle (Data Path) **4**
2.1 Question 1 4

3 Modification du chemin de contrôle (Control Path) **4**
3.1 Question 2 4
3.2 Question 3 5

4 Programmation d'une boucle **6**
4.1 Question 4 6
4.2 Question 5 6
4.3 Question 6 7

1 Introduction et mise en contexte

Dans ce TD, nous allons étendre notre chemin de données pour permettre l'exécution d'instructions de branchement. Pour ce faire, nous ajouterons les composants manquants nécessaires à l'exécution efficace de ces instructions. Nous intégrerons un multiplexeur pour sélectionner la future valeur du compteur programme (pc_next_s) et un port d'entrée pour le piloter (pc_next_sel_i). De plus, nous ajouterons un additionneur pour calculer l'adresse de branchement. En parallèle, nous apporterons des modifications à notre générateur de valeur immédiate afin de lui permettre de gérer les instructions de branchement correctement. Ces ajustements nous permettront d'améliorer la fonctionnalité de notre microarchitecture, en lui permettant de traiter efficacement les instructions de branchement lors de l'exécution des programmes.

2 Modification du chemin de contrôle (Data Path)

2.1 Question 1

Le traitement requis pour gérer la valeur immédiate dans une instruction de branchement consiste à étendre ou à modifier le générateur de valeur immédiate actuel afin qu'il puisse produire des valeurs immédiates compatibles avec les instructions de branchement.

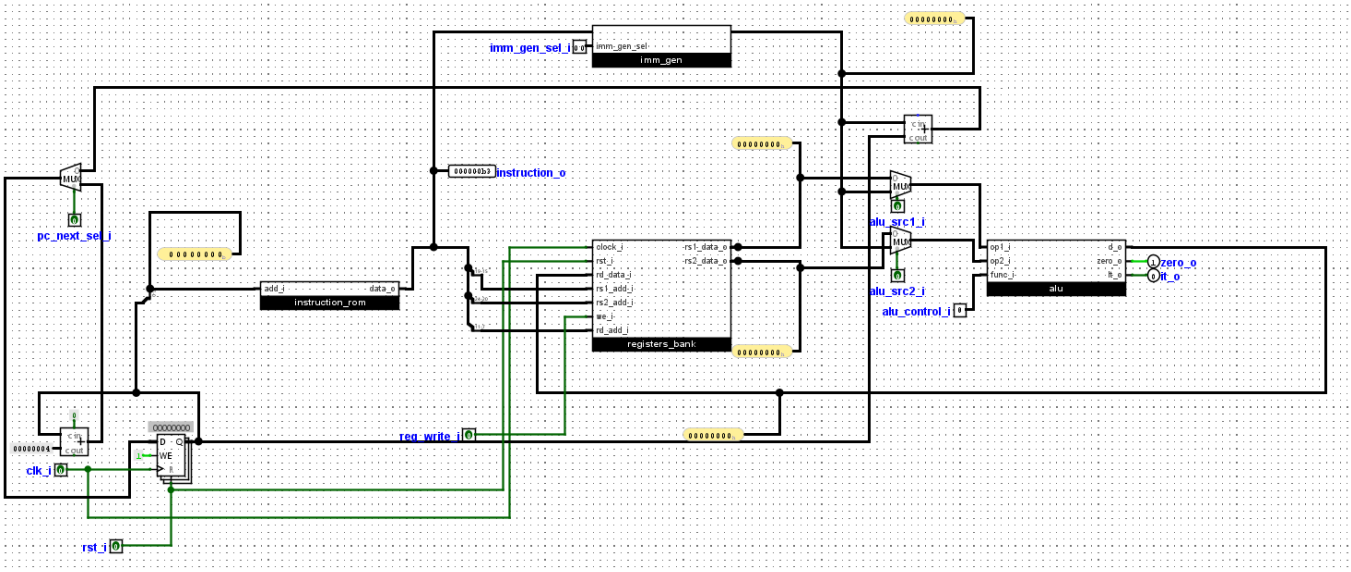


FIGURE 1 – Remplacement du bloc memory par le bloc instruction_rom

3 Modification du chemin de contrôle (Control Path)

Nous souhaitons implémenter l'instruction de branchement suivante :BEQ .

3.1 Question 2

On va utiliser le signal zero_o pour déterminer si la condition de branchement est remplie. En effet il indique si les 2 registres que l'on utilise pour la boucle sont égaux, afin de terminer cette boucle.

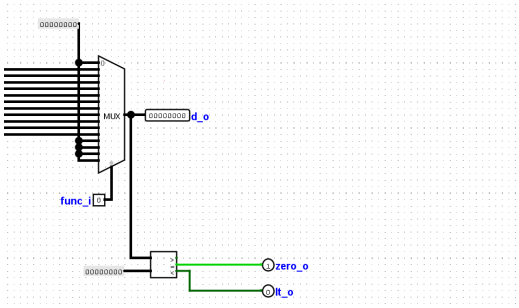
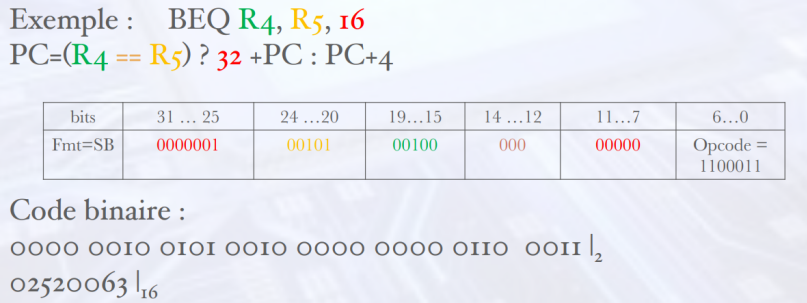


FIGURE 2 – Sortie de l'ALU

3.2 Question 3

L'opération qui va permettre à l'ALU d'activer ce signal suivant la condition requise par cette instruction de branchement est la soustraction. En effet, L'instruction BEQ permet à l'ALU de comparer deux valeurs dans les registres et d'effectuer un branchement conditionnel selon le résultat de la comparaison. Son objectif est de permettre au processeur de changer de direction vers une adresse différente si les valeurs dans les registres sont identiques. Concrètement, l'ALU examine les deux valeurs en entrée et, si elles correspondent, le processeur effectue un saut à l'adresse spécifiée par l'offset de branchement. Dans le cas contraire, l'exécution du programme se poursuit à l'adresse suivante.


Exemple : BEQ R4, R5, 16
PC=(R4 == R5) ? 32 +PC : PC+4

bits	31 … 25	24 …20	19…15	14 …12	11…7	6…0
Fmt=SB	0000001	00101	00100	000	00000	Opcode = 1100011

Code binaire :
0000 0010 0101 0010 0000 0000 0110 0011 $|_2$
02520063 $|_{16}$

FIGURE 3 – Instruction BEQ

4 Programmation d'une boucle

A l'aide des instructions ADDI et BEQ on peut programmer une boucle. Le compteur de boucle sera stocké dans R1. Le nombre d'itérations sera stocké dans R2, il sera de 10.

4.1 Question 4

Voici un programme assembleur correspondant :

$$ADDR1, R0, R0$$

$$ADDIR2, R0, OxA$$

$$ADDIR1, R0, Ox1$$

$$BNER1, R2, -2$$

4.2 Question 5

En assemblant le programme précédent et en effectuant la simulation, on obtient :

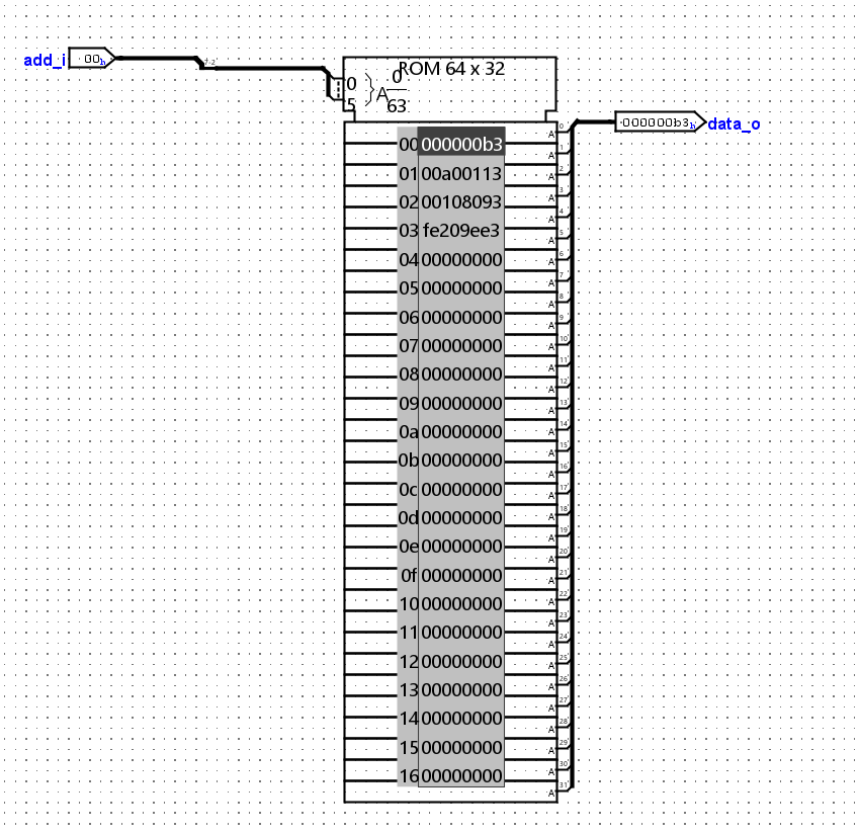


FIGURE 4 – Mémoire d'instructions ROM

4.3 Question 6

En exécutant le programme, on obtient les résultats suivants :

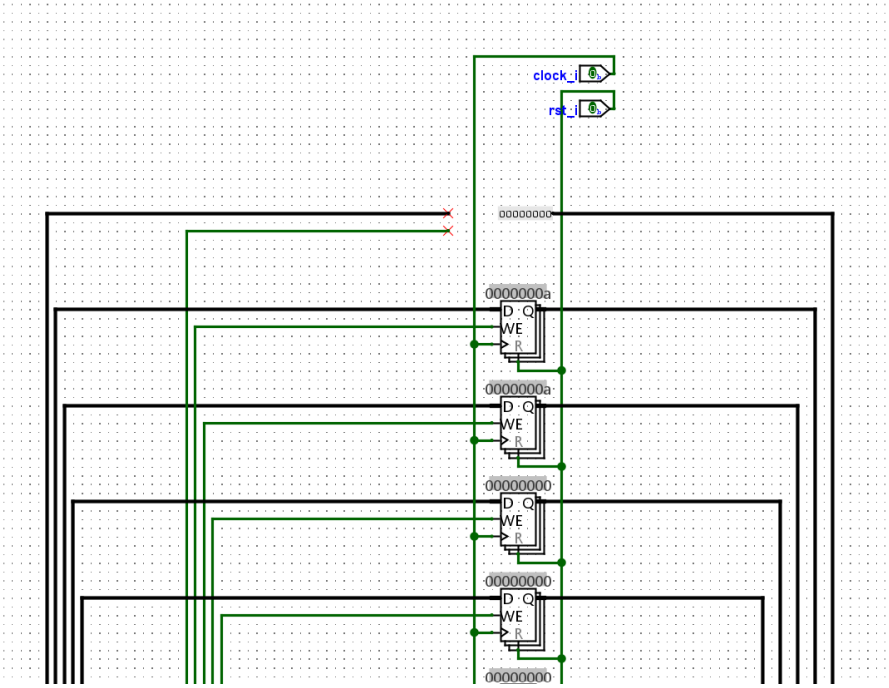


FIGURE 5 – Mémoire d'instructions ROM

Lorsque l'on a R1=R2, on arrête de réaliser des sauts dans le bloc mémoire. Ici on obtient cette condition lorsque R1=R2=10=A.
On peut observer le chronogramme suivant :

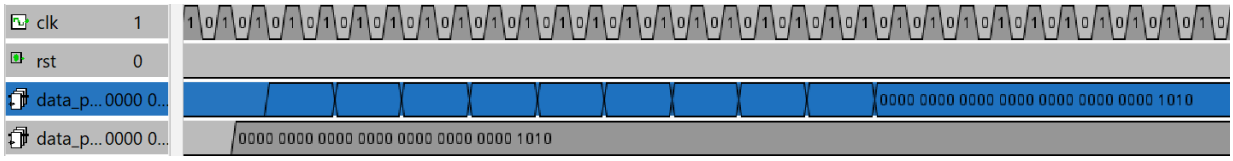


FIGURE 6 – Chronogramme associé au programme

***

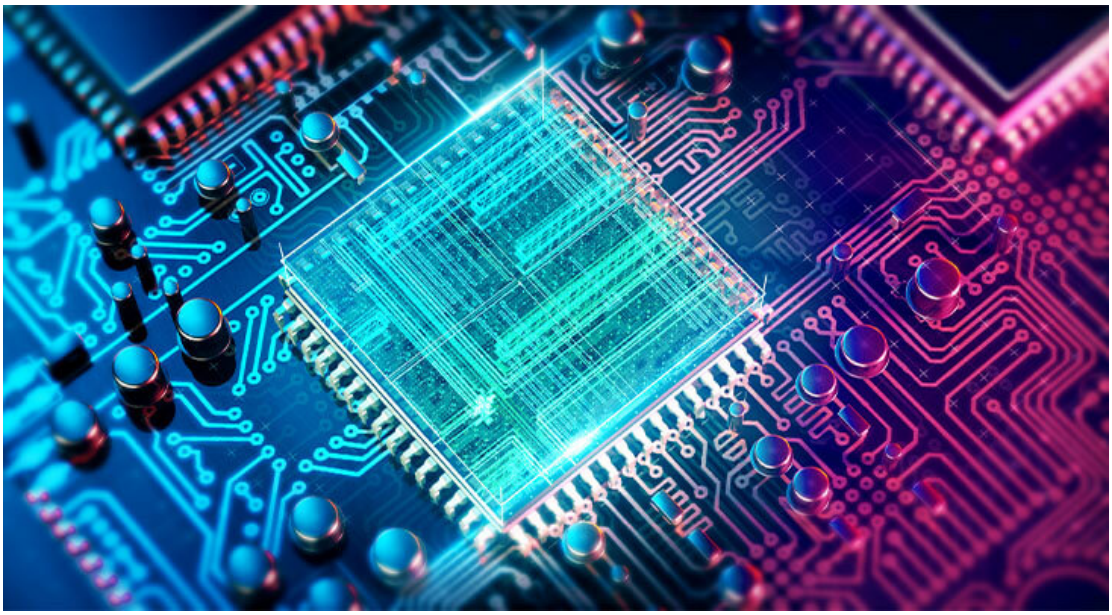


FIGURE 7

FIN DU RAPPORT

Merci pour votre attention !

TALEB Samy, VEYRAT Ylies